JAVA PROJECT . DESIGN GUIDE

APPARAT

Resources, Validation, CSV and Reports

Instrument management, file handling, utilization report

Shourya Goyal

24BBS0132

WHAT THIS DOCUMENT GIVES YOU

- What was promised in Review 1 and what exists now
- How the management, import and report flows work
- File handling, CSV, collections and streams explained simply
- A demo script, the tests, and honest limits
- Fourteen likely viva questions with model answers

Every number in this guide was measured on the project (tests run, requests sent, files counted).

1. Your slice at a glance

In Review 1 your slice was **Resources, Validation and Reports**. This table compares each promise with what the project contains today.

Review 1 item	Status	Where it lives
Resource management	**Done**	add, rename, change location, activate and deactivate (technicians and administrators)
Resource status	**Done**	ACTIVE / INACTIVE; inactive instruments are hidden from the booking form
Maintenance windows and maintenance UI	**Done**	technician page /manage/maintenance; windows block bookings for a time period; the availability page shows them as Maintenance
File handling	**Done**	CSV import (upload, read line by line) and CSV downloads (report and my bookings)
CSV import / export	**Done**	ImportResourcesServlet, ExportBookingsServlet, ReportServlet?format=csv
Report generation	**Done**	utilization report per instrument over a date range, as a page and a CSV file
Input validation	**Done**	four layers: browser, servlet, service, model constructor
Exception handling and testing	**Done**	typed exceptions mapped to HTTP codes; 38 new tests for this slice, including a concurrency test

2. The three flows

Managing instruments (add, rename, change status)

Importing instruments from CSV

Report and downloads

Maintenance windows

All four pass through **AuthFilter**: /manage and /reports need staff permissions (technician or administrator), so a student gets a 403 page (measured).

3. Files you own

File	Job	Lines
servlet/ManageResourcesServlet.java	list, add, update, status change; one-time messages	158
servlet/ImportResourcesServlet.java	receives the CSV upload and reads it	106
servlet/ReportServlet.java	report page and CSV download	123
servlet/ExportBookingsServlet.java	downloads the user's own bookings as CSV	67
servlet/ManageMaintenanceServlet.java	list, add and remove maintenance windows	161
service/MaintenanceService.java	maintenance rules: validation, lock, overlap checks	132
service/MaintenanceConflictException.java	window clashes with a booking or another window	8
service/MaintenanceNotFoundException.java	unknown window id	8
dao/MaintenanceDao.java	insert, delete, overlap queries	226
model/MaintenanceWindow.java	a window; the constructor rejects bad values	62
dto/MaintenanceView.java	a window with the instrument name, for the page	14
webapp/WEB-INF/views/manage-maintenance.jsp	maintenance page	125
service/ResourceAdminService.java	all validation, duplicate checks and import logic	264
service/ReportService.java	turns slot counts into hours and percentages	86
service/ResourceNotFoundException.java	unknown instrument id	8
service/DuplicateResourceException.java	name already used	8
dao/ResourceAdminDao.java	insert, update, status update	90
dao/ReportDao.java	usage counts per instrument	66
dto/ImportResult.java	rows imported plus one message per rejected row	12
dto/UsageCount.java	raw counts from the database	11
dto/UtilizationRow.java	one finished report line	12
util/CsvUtil.java	writes and parses CSV safely	107
webapp/WEB-INF/views/manage-resources.jsp	management page with add, import and edit forms	149
webapp/WEB-INF/views/report.jsp	report page	88
Total		**2091**

4. File handling, explained simply

Reading an upload

The browser sends the file inside the POST request (`multipart/form-data`). The servlet is marked `@MultipartConfig` with a 1 MB limit, and gets the file with `request.getPart("file")`. It must be non-empty and its name must end in .csv.

Reading line by line

A `BufferedReader` wraps an `InputStreamReader` that decodes the bytes as UTF-8. `readLine()` returns one line at a time, so even a large file never has to sit in memory as one block.

try-with-resources

The reader is opened in the try header, so it is closed automatically, even if an exception is thrown halfway through the file.

Writing a download

The servlet sets `Content-Type: text/csv` and `Content-Disposition: attachment`, so the browser saves a file instead of showing text. It then prints lines with a `PrintWriter`.

Byte-order mark

Excel sometimes puts an invisible character at the start of a UTF-8 file. The importer removes it, so the first column name is read correctly (tested).

5. CSV rules

Format: `name,location[,status]`. A header line is allowed. Status is optional (ACTIVE or INACTIVE, any letter case; default ACTIVE). Maximum 500 rows.

Example, and what the real run produced:

```
name,location,status
ZZ-VERIFY-Csv1,Bench 1
ZZ-VERIFY-Csv2,Bench 2,INACTIVE
ZZ-VERIFY-Csv1,Bench 3          <- duplicate name, rejected
"ZZ-VERIFY-Csv3, quoted",Bench 4  <- comma inside quotes, accepted
broken-line                     <- only one column, rejected
```

Result shown on the page: **Imported 3 instrument(s); 2 row(s) rejected**, followed by the two rejected line numbers with reasons.

- **Quoting.** A value containing a comma or quote is wrapped in quotes, and a quote inside is doubled. `CsvUtil` does this when writing and understands it when reading. Writing then reading gives back the original values (tested).
- **CSV injection.** A cell starting with = + - or @ can run as a formula when the file is opened in a spreadsheet. `CsvUtil.escape` puts a quote in front of such cells, so an instrument named =HYPERLINK(...) is exported as harmless text.
- **Duplicates.** Names are compared ignoring case, against instruments already stored *and* against earlier rows in the same file. The check uses a `Set`, which makes 'have I seen this name?' one quick lookup.
- **All valid rows or none of them.** Valid rows are inserted in **one transaction**. If the database fails halfway, everything is rolled back. Rejected rows never reach the database.

6. The utilization report

Definition: utilization = booked hours / open hours in the range. Open hours = number of days (both end dates included) x 9 hours (09:00 to 18:00). Booked hours = number of booked 30-minute slots x 0.5.

Worked example from the tests: one 10:00-11:00 booking (2 slots) on a single day gives 1.0 booked hour out of 9, which is **11.1 percent**. Over a two-day range it is 1 out of 18, which is **5.6 percent**.

The data comes from one SQL query. `resources LEFT JOIN booking_slots` keeps instruments that have no bookings (they show 0), and `COUNT(DISTINCT booking_id)` counts bookings while `COUNT(slot_start)` counts slots.

Why cancelled bookings do not count: cancelling deletes a booking's slot rows, so they are simply not there to be counted. Nothing special is needed in the report.

The service uses a **stream** to turn each database row into a report row, and the servlet uses streams to add up the footer totals.

7. Maintenance windows

What it is: a time period when an instrument cannot be booked, for example 10:00 to 11:00 for a lamp change. A booking that overlaps a window is refused with a clear message, and the availability page shows those slots as **Maintenance**. Long outages still use the INACTIVE switch.

Rules: a window follows the booking rules (one day, start and end on the half hour, inside 09:00 to 18:00, at least 30 minutes) and needs a reason of at most 200 characters. A window is refused if it overlaps another window, or if a **confirmed** booking already overlaps it (cancelled bookings do not count). Nothing is cancelled automatically.

Overlap test: two periods overlap when each starts before the other ends: `existing.start < new.end AND existing.end > new.start`. A window ending at 11:00 and a booking starting at 11:00 do not overlap (tested).

Why not reuse the booking_slots table? Its `booking_id` column is required and points to a booking, so a window cannot live there without changing that design.

How a booking and a window are kept apart: both `createBooking` and `addWindow` start a transaction, lock the instrument's row with `SELECT ... FOR UPDATE`, and only then check for overlaps. Only one of them can hold that lock at a time, so they take turns, and whoever goes second sees the other's result. This is the same lock that already removed the deadlocks in the booking stress test.

Proof: a test starts one thread adding a window and 29 threads booking overlapping times, and repeats that for 8 rounds. In the recorded run, the bookings won 6 rounds (the window was refused) and the window won 2 rounds (every booking was refused); the split changes from run to run. In every round a window and a booking never coexisted, and the only failures were the clean, expected types (no deadlocks). Removing the check from `createBooking` made this test fail, so it is a real guard.

Small changes to code written earlier: BookingService got one lookup and a four-line check after the 'instrument is active' check, and BookingServlet got one extra catch (409 with the maintenance message). database/setup.sql contains the new table.

8. Exceptions and HTTP codes

Situation	Exception	Status	User sees
Name already used	DuplicateResourceException	409	that name already exists
Window or booking overlaps the time	MaintenanceConflictException	409	the specific message
Unknown instrument or window id	ResourceNotFoundException / MaintenanceNotFoundException	404	does not exist
Blank, too long, or malformed value	IllegalArgumentException	400	the specific message
id or date is not a number or date	NumberFormatException / DateTimeParseException	400	please check your inputs
Wrong file type, empty file, file over 1 MB	(checked in the servlet)	400	message on the error page
Database failure	SQLException	500	generic error page, no details

9. Validation: four layers

Layer	What it checks	Why it is there
Browser (js/app.js)	required fields, length, .csv extension, 1 MB, start date not after end date	instant feedback
Servlet	action name, numeric ids, file present and named .csv	the request shape
Service	trimmed values, length 100, duplicate names, instrument exists, date range at most 366 days	business rules
Model (Resource constructor)	name, location and status not blank or null	an invalid object cannot exist

10. Demo script (six minutes)

- Log in as **tech@apparat.test** (password **Tech@123**, sample data) and open **Manage**.
- Add an instrument. Add the same name in capitals: the duplicate message appears.
- Rename it, then deactivate it, and open the booking form as a student: the instrument is not offered.
- Import a small CSV with one good row and one bad row: the summary lists the rejected line.
- Open **Reports**, choose a range, then press **Download CSV** and open the file.
- Open **Maintenance**, add a window 10:00-11:00 for an instrument with a reason.
- As a student, try to book 10:30-11:30 on that day: the booking is refused with the maintenance message. Open Availability: those slots show Maintenance.
- Log in as a student and try /reports: a friendly 'Not allowed' page.

11. What was tested

Test class	Tests	What it proves
CsvUtilTests	9	quoting, doubled quotes, formula neutralising, parsing of quoted fields, empty fields, missing closing quote, round trip
ResourceAdminServiceDbTests	11	add, duplicates ignoring case, blank and over-long values, rename, unknown id, status switch both ways, import with good and bad rows, existing names skipped, byte-order mark (real database, cleaned up afterwards)
ReportServiceDbTests	4	11.1 and 5.6 percent examples, zero rows for empty ranges, invalid ranges refused (creates a real booking, then removes it)
MaintenanceServiceDbTests	13	window saved and listed; booking inside or around a window refused with nothing saved; bookings that only touch the window allowed; window over a confirmed booking refused; cancelled bookings do not block; overlapping windows refused, adjacent ones allowed; invalid times and reasons; unknown instrument; removing a window reopens the time; unknown window; availability shows maintenance, booked and free slots
MaintenanceConcurrencyTests	1	8 rounds of one window against 29 bookings: a window and a booking never coexist, only expected failure types

The whole project suite is now **154 tests, 0 failures**. As a check that the new tests can fail, five deliberate bugs were put into the code one at a time (no duplicate check in the import, a wrong capacity in the report, a password check that always passes, bookings ignoring maintenance windows, windows ignoring existing bookings). Each was caught by the matching tests, and the code was then restored.

Checks made against the running site: adding, duplicate (409), blank (400), renaming, deactivating (and the booking form hiding it), unknown id (404), a name containing HTML shown as escaped text, a CSV import with 3 good and 2 bad rows, a .txt upload (400), a missing file (400), a 1.5 MB upload (400), the report page, the CSV download with the right headers, a reversed date range (400), and a student refused (403). For maintenance: adding a window, a booking inside it refused (409, nothing saved), availability marking two Maintenance rows, a window over an existing booking refused, an overlapping window refused, off-grid time and blank reason refused (400), removing a window and removing it again (404).

12. Honest limits (say these before the panel asks)

- **Maintenance windows cover part of one day only** (same rules as bookings). A longer outage uses the INACTIVE switch.
- **Adding a window never cancels bookings.** If confirmed bookings overlap, the window is refused and staff must cancel them first.
- A window cannot be edited; remove it and add a new one. Windows in the past are not rejected, because bookings have no 'future only' rule either.
- **Duplicate names are checked in Java only.** The table has no unique constraint on the name, so two staff members adding the same name at the exact same moment could both succeed. A unique constraint would close this; it is not added yet.
- Deactivating an instrument does not cancel bookings that already exist for it.
- Import has no preview step: valid rows are saved immediately.
- The report counts confirmed bookings through their slots; it has no charts and no per-user breakdown.
- Staff pages are open to technicians and administrators alike; there is no way to give one person only part of them.

13. Viva questions with model answers

Q1. How does the import read a file?

The servlet gets the uploaded part, wraps its input stream in an InputStreamReader (UTF-8) and a BufferedReader, and reads one line at a time inside try-with-resources, so the file is always closed.

Q2. Why try-with-resources?

It closes the file automatically when the block ends, even if an exception is thrown. Forgetting to close a file leaks a resource.

Q3. What happens to bad rows in an import?

Each bad row is skipped and reported with its line number and reason. Valid rows are saved together in one transaction, so the database never ends up half imported.

Q4. Why a transaction for the import?

To make saving the valid rows all-or-nothing. If row 40 fails in the database, rows 1 to 39 are rolled back too.

Q5. What is CSV injection and how do you prevent it?

A cell beginning with =, +, - or @ can run as a formula in a spreadsheet. CsvUtil.escape puts a quote before such a cell so it is read as text.

Q6. Why use a Set for duplicate names?

A set answers 'have I seen this?' with one lookup, and add() tells us in the same step whether the name was new. A list would need a scan for every row.

Q7. How is utilization calculated?

Booked hours (slots x 0.5) divided by open hours (days x 9). One booked hour in one day is 1/9, which is 11.1 percent.

Q8. Why does the report use a LEFT JOIN?

So an instrument with no bookings still appears, with zeros. An inner join would silently drop it.

Q9. Why do cancelled bookings not count?

Cancelling deletes the booking's slot rows, and the report counts slot rows.

Q10. Where is validation done, and why in several places?

In the browser (for speed), the servlet, the service (business rules) and the model constructor. The browser can be bypassed, so the server layers are the real protection.

Q11. Which exception becomes which HTTP code?

Duplicate name 409, unknown instrument 404, bad input 400, database failure 500. Each type lets the servlet choose the right response.

Q12. How do you stop a booking and a maintenance window being saved for the same time?

Both transactions lock the instrument's row with SELECT ... FOR UPDATE and check for overlaps only after getting that lock, so they take turns. A test races one window against 29 bookings for 8 rounds, and a window and a booking never coexisted.

Q13. Why was the booking_slots table not reused for maintenance?

Its booking_id column is required and refers to a booking, so a maintenance window would need a different table design. A separate table plus the instrument lock gives the same safety.

Q14. What is the weakest part of this slice?

Duplicate instrument names are only checked in Java, so a unique constraint on the name is the fix. Maintenance windows are limited to one day and cannot be edited.